
1. What is Software Quality Assurance (SQA)?

SQA is the **process-oriented** side of quality management.

- **Definition:** A planned and systematic pattern of all actions necessary to provide adequate confidence that an item or product conforms to established technical requirements.
- **The SQA Mindset:** SQA is about **prevention**, not just detection. It focuses on the *process* used to create the software rather than just the software itself.
- **Core SQA Activities:**
 - Setting standards (coding standards, documentation templates).
 - Performing Technical Reviews (Inspections and Walkthroughs).
 - Auditing processes to ensure compliance.
 - Analyzing defects to improve the future process.

2. The Relationship: SQA vs. Software Testing (ST)

- **SQA (The Umbrella):** SQA is the broad set of activities that ensure the process is healthy. It is proactive.
- **Software Testing (The Specialist):** Testing is a subset of SQA. It is **product-oriented** and reactive. Testing is the actual execution of software to find defects.

Feature	Software Quality Assurance (SQA)	Software Testing (ST)
Orientation	Process-oriented	Product-oriented

Goal	Prevent defects	Find/detect defects
Nature	Proactive (planning/auditing)	Reactive (execution)
Scope	Entire SDLC	Testing Phase (SIT, UAT, etc.)

3. How Testing Supports SQA

Testing provides the **data** that SQA needs to improve the process.

1. **Verification:** Testing checks if we are building the product correctly (conforming to the SQA plan).
2. **Validation:** Testing checks if we are building the right product (meeting user needs).
3. **Metrics for SQA:** Testing results (like **Defect Arrival Rate** and **Defect Density**) allow SQA managers to see if the development process is stable or if the team needs better training/standards.

4. The SQA Process Steps

ISO standard software quality assurance (SQA) processes, often aligned with ISO 9001 and ISO/IEC 25010 (formerly 9126), define structured methodologies to ensure software meets requirements through planning, technical reviews, testing, and continuous improvement. The process focuses on establishing a quality management system (QMS), auditing compliance, and evaluating six key characteristics: functionality, reliability, usability, efficiency, maintainability, and portability

5. The SQA Process Steps

1. **Requirement Review:** Ensuring quality starts at the "Definition" phase.
 2. **Design Inspection:** Checking the architecture before a single line of code is written.
 3. **Standards Compliance:** Ensuring the developers follow the agreed-upon SQA guidelines.
 4. **Verification & Validation (Testing):** Executing the test cases.
 5. **Audit & Feedback:** Comparing the final results against the initial SQA goals to improve the next release.
-

6. Elements of SQA

- **Standards:** The IEEE, ISO, and other standards organizations have produced a broad array of software engineering standards and related documents. The job of SQA is to ensure that standards that have been adopted are followed and that all work products conform to them.
- **Reviews and audits:** Technical reviews are a quality control activity performed by software engineers for software engineers. Their intent is to uncover errors. Audits are a type of review performed by SQA personnel (people employed in an organization) with the intent of ensuring that quality guidelines are being followed for software engineering work.
- **Testing:** [Software testing](#) is a quality control function that has one primary goal to find errors. The job of SQA is to ensure that testing

is properly planned and efficiently conducted for primary goal of software.

- **Error/defect collection and analysis** : SQA collects and analyzes error and defect data to better understand how errors are introduced and what software engineering activities are best suited to eliminating them.
- **Change management**: SQA ensures that adequate change management practices have been instituted.

SQA is not just about finding bugs in the code; it is about ensuring that the **process** of modifying that code is controlled and documented.

"Change Management" refers to the formal procedure followed when a requirement, design, or piece of code needs to be altered.

- **Education**: Every software organization wants to improve its software engineering practices. A key contributor to improvement is education of software engineers, their managers, and other stakeholders. The SQA organization takes the lead in software process improvement which is key proponent and sponsor of educational programs.
- **Security management**: SQA ensures that appropriate process and technology are used to achieve software security.

In SQA, security is treated as a process requirement rather than just a

feature. SQA's role is to verify that the development team isn't just "coding" security but is following a **Secured SDLC**.

Process Oversight: SQA ensures that activities like security design reviews and threat modeling are integrated into the workflow.

Technology Verification: SQA audits the use of specific technologies, such as ensuring data encryption standards are met and that automated vulnerability scanning tools (like Static Analysis) are effectively utilized by the developers.

- **Safety:** SQA may be responsible for assessing the impact of software failure and for initiating those steps required to reduce risk.

While security protects the software from outside threats, **Safety** focuses on protecting the environment or users from the software itself. This is critical in domains like healthcare or aerospace.

Impact Assessment: SQA identifies "safety-critical" modules where a single failure could lead to catastrophic loss (injury or property damage).

Risk Reduction: SQA ensures that fail-safe mechanisms are designed into the software. For example, if a sensor fails, SQA verifies that the process requires the software to enter a "Safe

State" (like shutting down a heater) rather than continuing with corrupted data.

- **Risk management:** The SQA organization ensures that risk management activities are properly conducted and that risk-related contingency plans have been established.

SQA acts as the auditor of the project's risk profile. It is the responsibility of SQA to ensure that risks are not just "identified" at the start of the project but are managed throughout the lifecycle.

Activity Oversight: SQA verifies that the project manager is regularly updating the **Risk Register** and that risk mitigation activities (like early prototyping for complex features) are actually happening.

Contingency Planning: SQA ensures that "Plan B" exists for high-impact risks. If a key technology or vendor fails, SQA checks that a documented contingency plan is ready to be triggered, ensuring the project's quality and timeline remain stable.

7. Focus of SQA



8. Activities of SQA

- **SQA Management Plan:** Make a plan for how you will carry out the SQA throughout the project. Think about which set of software engineering activities are the best for project. check level of SQA team skills.
- **Set The Check Points:** SQA team should set checkpoints. Evaluate the performance of the project on the basis of collected data on different check points.
- **Measure Change Impact:** The changes for making the correction of an error sometimes re introduces more errors keep the measure of impact of change on project. Reset the new change to check the compatibility of this fix with whole project.
- **Multi testing Strategy:** Do not depend on a single testing approach. When you have a lot of testing approaches available use them.

- **Manage Good Relations:** In the working environment managing good relations with other teams involved in the project development is mandatory. Bad relation of SQA team with programmers team will impact directly and badly on project.
- **Maintaining records and reports:** Comprehensively document and share all QA records, including test cases, defects, changes, and cycles, for stakeholder awareness and future reference.
- **Reviews software engineering activities:** The SQA group identifies and documents the processes. The group also verifies the correctness of software product.
- **Formalize deviation handling:** Track and document software deviations meticulously. Follows established procedures for handling variances.

If a developer realizes a specific module cannot meet the speed requirements, they cannot just ignore it. Under **Formalized Deviation Handling**, they must document this variance. SQA then ensures that either the code is optimized to meet the goal or the requirement is formally changed through the **Change Management** process we discussed earlier.

9. Benefits of Software Quality Assurance (SQA)

- SQA produces high quality software.
- High quality application saves time and cost.
- SQA is beneficial for better reliability.

- SQA is beneficial in the condition of no maintenance for a long time.
- High quality commercial software increase market share of company.
- Improving the process of creating software.
- Improves the quality of the software.
- It cuts maintenance costs. Get the release right the first time, and your company can forget about it and move on to the next big thing. Release a product with chronic issues, and your business bogs down in a costly, time consuming, never-ending cycle of repairs.

10. Disadvantage of Software Quality Assurance (SQA)

- **Cost:** Some of them include adding more resources, which cause the more budget its not, addition of more resources for betterment of the product.
- **Time Consuming:** Testing and Deployment of the project taking more time which cause delay in the project.
- **Overhead :** SQA processes can introduce administrative overhead, requiring documentation, reporting, and tracking of quality metrics. This additional administrative burden can sometimes outweigh the benefits, especially for smaller projects.
- **Resource Intensive :** SQA requires skilled personnel with expertise in testing methodologies, tools, and quality assurance practices.

Acquiring and retaining such talent can be challenging and expensive.

- **Resistance to Change** : Some team members may resist the implementation of SQA processes, viewing them as bureaucratic or unnecessary. This resistance can hinder the adoption and effectiveness of quality assurance practices within an organization.
- **Not Foolproof** : Despite thorough testing and quality assurance efforts, software can still contain defects or vulnerabilities. SQA cannot guarantee the elimination of all bugs or issues in software products.
- **Complexity** : SQA processes can be complex, especially in large-scale projects with multiple stakeholders, dependencies, and integration points. Managing the complexity of quality assurance activities requires careful planning and coordination.

11. SQA Standards: There are several internationally recognized standards that define a formal Software Quality Assurance (SQA) process. While organizations often customize their workflows, the most widely accepted "standard" frameworks come from the IEEE and ISO.

While universal standards like IEEE, ISO, and CMMI exist to provide a shared language of quality, they are **architectural frameworks**.

In the real world, no two SQA processes look identical because every supplier must **tailor** these high-level requirements into a unique 'Amalgam' that fits their specific culture, project scale, and customer needs."

1. The IEEE 730 Standard

The **IEEE 730** is the most specific industry standard for Software Quality Assurance Plans. It defines a set of mandatory activities that must be performed to ensure a software product meets its requirements.

According to this standard, a formal SQA process must include:

- **Software Quality Assurance Planning:** Creating a roadmap that defines the standards, tools, and methodologies to be used.
- **Reviews and Audits:** * **Managerial Reviews:** Assessing the execution of the SQA plan.
 - **Technical Reviews:** Evaluating the software products (requirements, design, code).
 - **Functional/Physical Audits:** Verifying that the software meets all functional requirements and is technically complete.
- **Documentation Oversight:** Ensuring that all project documents (user manuals, design specs) are accurate and controlled.
- **Process Monitoring:** Checking that the development team is actually following the defined SDLC (Software Development Life Cycle).

2. The ISO 9000 / 9001:2000 Framework

Many academic and professional programs use the **ISO 9001** framework (as detailed in standard textbooks like Naik & Tripathy) to define the SQA process. This framework focuses on a **Quality Management System (QMS)** rather than just a checklist.

The standard ISO-based SQA process follows these steps:

1. **Establish Quality Policy:** Define the quality goals for the organization.
2. **Define Processes:** Standardize how requirements are gathered, how code is written, and how testing is performed.
3. **Quality Control (Testing):** Execute tests to identify defects (this is the reactive part of the process).
4. **Quality Audit:** An independent team checks if the project is adhering to the established processes.
5. **Corrective Action:** If a deviation is found (e.g., a review was skipped), the process is adjusted to prevent it from happening again.

3. The General SQA Lifecycle (Industry Practice)

In a typical software house, the "standard" SQA process usually follows this simplified lifecycle:

- **Step 1: SQA Planning:** Done during the requirements phase. Defines "What is quality for this project?"
- **Step 2: Standards Setting:** Defining coding standards, naming conventions, and documentation templates.
- **Step 3: Verification (Static Testing):** Conducting walkthroughs, inspections, and reviews of the design and code before it runs.
- **Step 4: Validation (Dynamic Testing):** The actual execution of software testing (Unit, Integration, System).
- **Step 5: Change and Deviation Management:** Managing how bugs are reported, how changes are approved, and how risks are mitigated.
- **Step 6: Reporting and Release Sign-off:** The SQA lead provides a "Quality Gate" report to decide if the software is ready for the customer.

Process Comparison

Standard	Focus Area	Best Used For...
IEEE 730	Specific SQA Activities	Highly regulated or mission-critical software.
ISO 9001	Organizational, Mmngt	Standardizing the entire company's workflow.
CMMI	Process Maturity	Organizations wanting to improve their capability over time.

12. SQA Plan and Project Plan:

1. Introduction: The Power of Planning

- **Key Concept:** Planning is the cornerstone of success in software development and quality assurance.
 - **The Two Pillars:** * **SQA Plan (SQAP):** Focuses on the activities and resources of the SQA function for the upcoming year.
 - **Project Plan:** Focuses on the activities, schedule, and resources for the project team throughout the software life cycle.
 - **Relationship:** The two plans must be tightly coordinated, as many SQA activities are mapped directly to project milestones.
-

2. The SQA Plan (SQAP)

The SQAP enables an organization to estimate the resources and professional expertise required to perform quality tasks.

A. Preparation Process

- The SQA function typically prepares the plan, which is then reviewed and approved by management.
- Estimates are based on project requirements, including staff days, skills, and required equipment.

B. Core Elements of SQAP

1. **Management Elements:** Organizational structure, roles, and responsibilities.
2. **Product Assurance Activities:**
 - **Conformance Evaluation of Project Plans:** Checking if plans meet contracts and standards.

- **Conformance Evaluation of Products:** Reviewing design reports, V&V, and test reports.
 - **Evaluation for Acceptability:** Ensuring the product meets the acquirer's confidence levels.
 - 3. **Process Assurance Activities:** Evaluating life cycle processes, the development environment, subcontractor performance, and staff competence.
-

3. The Project Plan

The project manager is responsible for this plan, which must be completed and reviewed before implementation begins.

A. Objectives

1. Ensure timely performance and compliance with contracts/standards.
2. Ensure high product quality.
3. Provide management with data for effective project control.

B. Development Plan Elements

A comprehensive development plan includes:

- **Project Products:** Defining what will be delivered (code, documentation).
- **Project Interfaces:** Identifying internal and external dependencies.
- **Project Schedule & Milestones:** Key events like the completion of the design phase.
- **Resource Estimates:** Human resources, equipment, and budget.
- **Risk Management:** Identifying product and development risks.
- **Methodology & Tools:** The specific tools and processes for each phase.
- **Standards & Procedures:** List of coding standards or work instructions to be applied.

C. Quality Plan Elements

The Quality Plan is a subset of the Project Plan focused on:

- **Quality Goals:** Quantifiable targets for the product.
 - **Planned Review Activities:** Scope, type, and schedule for Design Reviews (DRs) and inspections.
 - **Planned Software Tests:** Unit, integration, system, and automated tests.
 - **Acceptance Tests for External Software:** For COTS, subcontractor code, or customer-supplied software.
 - **Configuration Management:** Tools and procedures for version control.
-

4. Special Cases: Small and Internal Projects

A. Small Projects (<15 man-days)

- Plans are often optional but highly recommended.
- **Benefits:** Better task understanding, improved commitment to obligations, and easier management control.

B. Internal Projects

- These are projects for use within the organization or for the general market (COTS).
 - **Risk:** Often "unplanned" due to the lack of an external customer, leading to budget overruns and delays.
 - **Recommendation:** Treat them as "regular projects" with full development and quality plans.
-

5. Software Risk Management (Appendix 7.A)

Risk management is a continuous process of identification, evaluation, and mitigation.

Risk Classification

- **Product Risks:** Physical, safety, or financial risks related to the software output.
- **Development Risks:** * **Requirement Risks:** Vague or changing specs.
 - **Team Member Risks:** Staff turnover or lack of skills.
 - **Estimation Risks:** Unrealistic budgets or schedules.

Questions:

📖 How does an updated project plan save a manager from "inevitable problems"? (Refer to the Jack mini-case) .

📖 Can a project proposal serve as a project plan? (Discuss the validity of proposal data after a contract is signed) .

Jack thanks his department manager– a mini case :

Jack Bora, an experienced project manager, has been appointed to manage the Pagoda project, a sizable project for automating a textile manufacturing plant. As is often the case in the software industry, due to an earlier obligation he could only begin organizing and executing the project 5 weeks after the scheduled project start. Naturally, he was under serious time pressure from the very first day. As he was a member of the proposal team and participated in most of the meetings held with the customer's representatives, he was confident that he knew everything that was required to do the job, and could even save himself the tiring task of preparing a project plan. He intended to use the proposal itself and the working papers as the project's development and quality plans. His reliance on these materials was based on the fact that he knew that the proposal and all its estimates, including the schedule, staff requirements, list of deliverables, software products, and list of development risks had all been thoroughly reviewed by the contract review team, and also amended accordingly. The first month of the Pagoda project was by now progressing well with no notable events. Then, one morning Jack was urgently called into the department manager's office. The development department manager did not waste a minute, and demanded that Jack immediately prepare new full project development

plans (“development plan”) and project quality plans (“quality plan”). When Jack tried to claim that there was no need for a project plan, as he had all the proposal team materials: working papers, including the project plan, resource estimates, list of project risks, and more. The department manager cut him short saying “I still insist you prepare entirely new project development and quality plans and complete them before the end of next week.” After a few moments of silence, he continued “By the way, don’t forget that a period of seven months elapsed between the proposal preparation and the signing of the contract. Such a period is a hell of time for changes in our trade. . . .” So, without actually agreeing regarding the need for development plans, Jack began carrying out “the manager’s task.” He reexamined the proposal materials and thoroughly updated them. The new plans were more comprehensive than the approved proposal, especially with respect to schedules, resource estimates, and development risk evaluations, and included additional subjects, which were not covered in the original proposal. The effort invested in preparing the development and quality plans proved to be beneficial:

He found out that the 5-week delay in the project start necessitates him to change the work sequence of the project to enable finishing on time. This change created a need for an additional team on the project.

- He soon discovered that two of the proposed team leaders would not be available at the newly scheduled dates, due to assignments recently allocated. Another team leader would be needed for the newly created team, so he would now need to locate and recruit three team leaders. (“Lucky I am to discover the team leader shortage, at a time when there is still a good chance to find a suitable candidate,” he thought).

- BGSoftware Consultants, the consulting company that had agreed to provide professional support in a highly specialized and crucial area, had suffered heavy losses and had consequently gone bankrupt 3 months earlier.

Jerry Fox, the company’s expert for the database chosen by the customer, had resigned a month ago.

- The human resource department notified Jack that only three out of the five programmers scheduled on the fourth to sixth month of the project could now be assigned for the project.

When he finally brought the development and quality plans to the department manager, he opened saying “I should have brought a bottle of wine to thank you for making me prepare updated plans. You have no doubt saved me a lot of inevitable problems managing the project.”